

MINICURSO WEBDEV

PET SAÚDE



QUEM SOU EU?

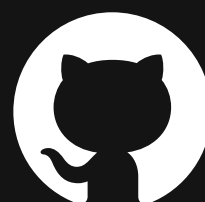


**DAVI
BITTENCOURT**

- Estudante de Sistemas de Informação - 6º período;
- Membro da LAWD desde 2024;
- Site do PS LAWD 2025 e novo Portal UFS;
- Aproximadamente 2 anos de experiência com desenvolvimento web.



<https://www.linkedin.com/in/davibital/>



<https://github.com/davibital>



O QUE IREMOS APRENDER?

- Como construir o **frontend** de aplicações web;
- React + TypeScript;
- Next.js (SabOOOr aprendido).

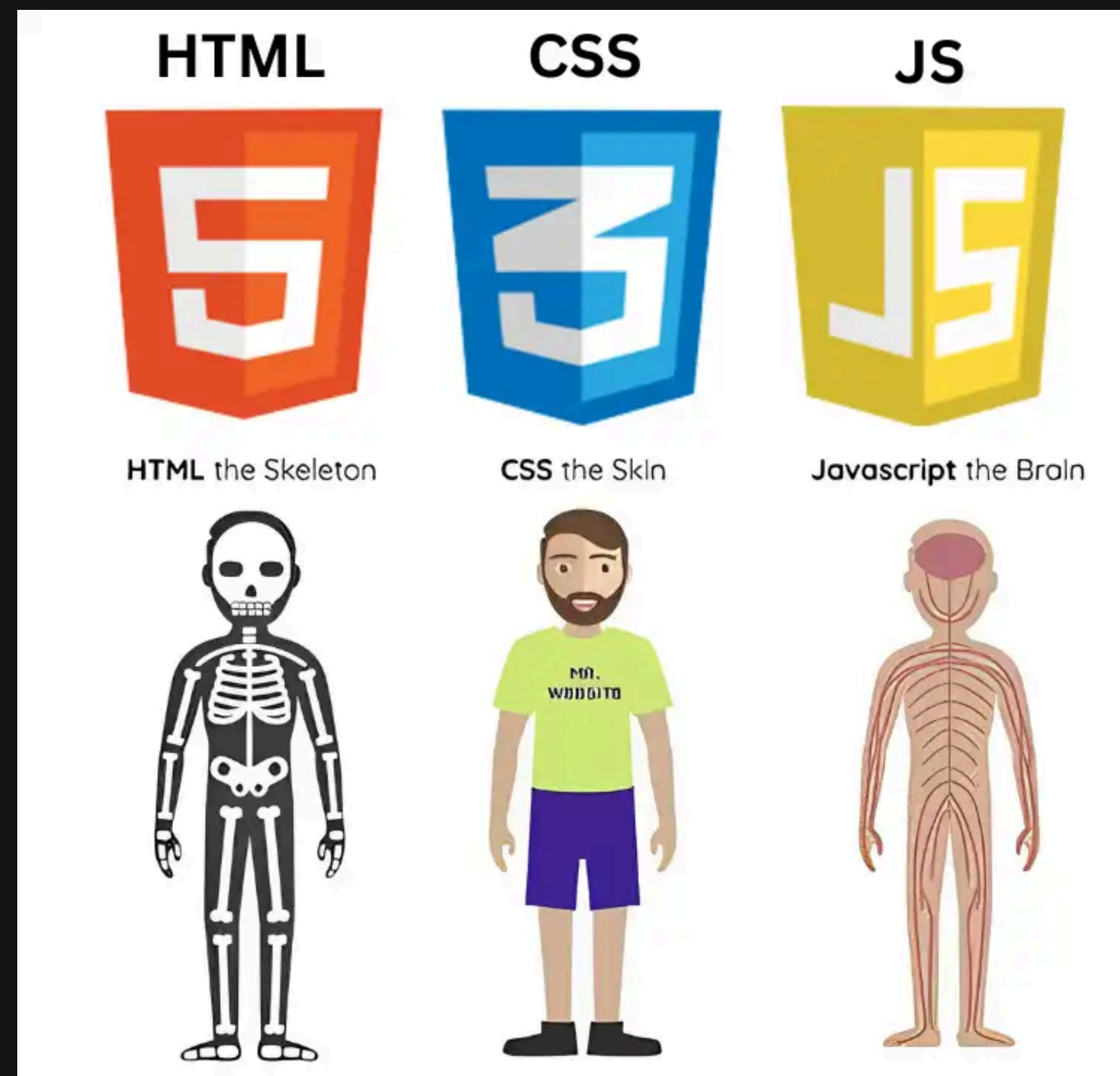
REPOSITÓRIO DO QITHUB

<https://github.com/davibital/minicurso-pet-saude>

`npm install`

`npm run dev`

ECOSSISTEMA WEB DO FRONTEND



ECOSSISTEMA WEB DO FRONTEND

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Minha página</title>
7   <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
10   <header>
11     <h1>Hello, world!</h1>
12   </header>
13   <main>
14     <h2>Esse é o conteúdo principal</h2>
15     <ul id="minha-lista">
16     </ul>
17     <button id="meu-botao">0 cliques</button>
18   </main>
19   <footer>
20     <p>Rodapé da página</p>
21   </footer>
22   <script src="script.js"></script>
23 </body>
24 </html>
```

ECOSSISTEMA WEB DO FRONTEND

```
1  #meu-botao {  
2      color: blue;  
3      /* color: rgb(0, 0 255); */  
4      width: 100px;  
5      height: 25px;  
6      border-radius: 10px;  
7  }  
8  
9  #meu-botao:hover {  
10     opacity: 0.8;  
11 }
```


ECOSSISTEMA WEB DO FRONTEND

```
1 const minhaLista = document.getElementById('minha-lista');
2 const meuBotao = document.getElementById('meu-botao');
3
4 let cliques = 0;
5 const itensLista = [
6   'Primeiro item',
7   'Segundo item',
8   'Terceiro item'
9 ];
10
11 function atualizarContadorClique(event) {
12   cliques += 1;
13
14   event.target.innerText = `${cliques} clique${cliques !== 1 ? 's' : ''}`;
15 }
16
17 meuBotao.addEventListener('click', atualizarContadorClique);
18
19 for (let i = 0; i < itensLista.length; i++)
20 {
21   const li = document.createElement('li');
22   li.innerText = itensLista[i];
23
24   minhaLista.appendChild(li);
25 }
```

ECOSSISTEMA WEB DO FRONTEND

Hello, world!

Esse é o conteúdo principal

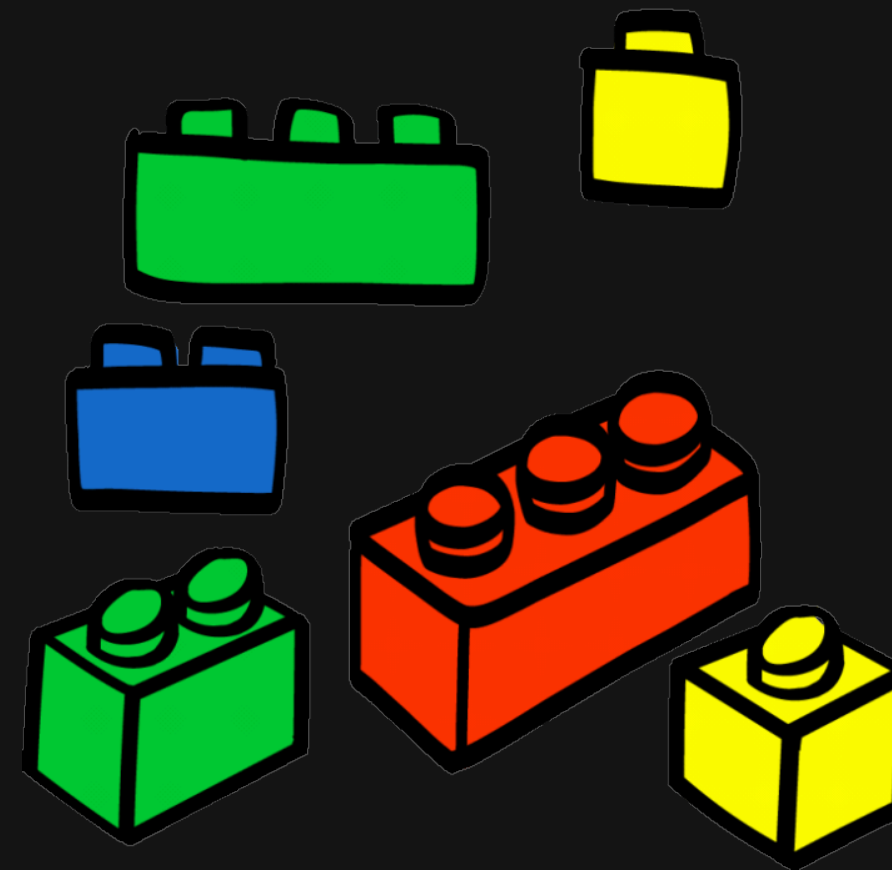
- Primeiro item
- Segundo item
- Terceiro item

2 cliques

Rodapé da página

POR QUE REACT?

- Componentização
- Reutilização
- Interface mais fluida

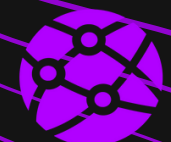


DO VANILLA PARA REACT

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Minha página</title>
7   <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
10   <header>
11     <h1>Hello, world!</h1>
12   </header>
13   <main>
14     <h2>Esse é o conteúdo principal</h2>
15     <ul id="minha-lista">
16     </ul>
17     <button id="meu-botao">0 cliques</button>
18   </main>
19   <footer>
20     <p>Rodapé da página</p>
21   </footer>
22   <script src="script.js"></script>
23 </body>
24 </html>
```

```
1 const minhaLista = document.getElementById('minha-lista');
2 const meuBotao = document.getElementById('meu-botao');
3
4 let cliques = 0;
5 const itensLista = [
6   'Primeiro item',
7   'Segundo item',
8   'Terceiro item'
9 ];
10
11 function atualizarContadorClique(event) {
12   cliques += 1;
13   event.target.innerText = `${cliques} clique${cliques !== 1 ? 's' : ''}`;
14 }
15
16 meuBotao.addEventListener('click', atualizarContadorClique);
17
18 for (let i = 0; i < itensLista.length; i++)
19 {
20   const li = document.createElement('li');
21   li.innerText = itensLista[i];
22   minhaLista.appendChild(li);
23 }
24
25 }
```

```
1 import styles from "../page.module.css";
2 import { Lista } from "@app/components/Lista";
3 import { BotaoContador } from "@app/components/BotaoContador";
4
5 export default function Home() {
6   const itensLista = [
7     'Primeiro item',
8     'Segundo item',
9     'Terceiro item'
10  ];
11
12   return (
13     <
14       <header>
15         <h1>Hello, world!</h1>
16       </header>
17       <main>
18         <h2>Esse é o conteúdo principal</h2>
19         <Lista items={itensLista} />
20         <BotaoContador className={styles.botao} />
21       </main>
22       <footer>
23         <p>Rodapé da página</p>
24       </footer>
25     </
26   );
27 }
```



DO VANILLA PARA REACT

```
1 const minhaLista = document.getElementById('minha-lista');
2 const meuBotao = document.getElementById('meu-botao');
3
4 let cliques = 0;
5 const itensLista = [
6   'Primeiro item',
7   'Segundo item',
8   'Terceiro item'
9 ];
10
11 function atualizarContadorClique(event) {
12   cliques += 1;
13   event.target.innerText = `${cliques} clique${cliques !== 1 ? 's' : ''}`;
14 }
15
16 meuBotao.addEventListener('click', atualizarContadorClique);
17
18 for (let i = 0; i < itensLista.length; i++)
19 {
20   const li = document.createElement('li');
21   li.innerText = itensLista[i];
22   minhaLista.appendChild(li);
23 }
24
25 }
```

```
1 export const Lista = ({ items }) => {
2   return (
3     <ul>
4       {items.map(item => <li>{item}</li>)}
5     </ul>
6   )
7 }
```

```
1 import { useState } from "react";
2
3 export const BotaoContador = ({ className }) => {
4   const [cliques, setCliques] = useState(0);
5
6   function atualizarClique() {
7     setCliques(prev => prev + 1);
8   }
9
10  const texto = `${cliques} clique${cliques !== 1 ? 's' : ''}`;
11
12  return (
13    <button onClick={atualizarClique} className={className}>
14      {texto}
15    </button>
16  );
17 }
```

ENTENDENDO O **JSX**

- Extensão da sintaxe **HTML**;
- Pode ser entendida como uma linguagem de template.

```
1 function formatName(user) {  
2   return user.firstName + ' ' + user.lastName;  
3 }  
4  
5 const user = {  
6   firstName: 'Harper',  
7   lastName: 'Perez'  
8 };  
9  
10 const element = (  
11   <h1>  
12     Hello, {formatName(user)}!  
13   </h1>  
14 );
```

POR QUE TYPESCRIPT?

- Adiciona robustês ao JavaScript através da **tipagem**;
- Melhora experiência de desenvolvimento;
- Integra ao editor de texto e **detecta erros**.


$$\text{JS} + \text{Tipos} = \text{TS}$$

POR QUE TYPESCRIPT?

No editor warnings
in JavaScript files

This code crashes at
runtime!

```
function compact(arr) {  
    if (arr.length > 10)  
        return arr.trim(0, 10)  
    return arr  
}
```

POR QUE TYPESCRIPT?

Adding this to a JS
file shows errors in
your editor

`// @ts-check`

```
function compact(arr) {  
  if (orr.length > 10)  
    return arr.trim(0, 10)  
  return arr  
}
```

Cannot find name 'orr'.

← the param is arr,
not orr!

POR QUE TYPESCRIPT?

Using JSDoc to give
type information

```
// @ts-check  
  
/** @param {any[]} arr */  
function compact(arr) {  
  if (arr.length > 10)  
    return arr.trim(0, 10)  
  return arr  
}
```

Property 'trim' does not exist on type 'any[]'.

Now TS has found a
bad call. Arrays
have slice, not
trim.

POR QUE TYPESCRIPT?

TypeScript adds
natural syntax for
providing types

```
function compact(arr: string[]) {  
  if (arr.length > 10)  
    return arr.slice(0, 10)  
  return arr  
}
```


REACT + TYPESCRIPT?

- Documentação viva dos componentes;
- Integração com o **JSX**;
- Segurança na renderização dos componentes.

REACT + TYPESCRIPT?

```
1 interface ListaProps {
2   items: string[];
3 }
4
5 // ou
6
7 type ListaProps = {
8   items: string[];
9 }
10
11 export const Lista = ({ items }: ListaProps) => {
12   return (
13     <ul>
14       {items.map(item => <li>{item}</li>)}
15     </ul>
16   )
17 }
```

VAMOS PRATICAR!

Qual **componente** posso criar para o meu projeto do **PET Saúde?**

Ele é **reutilizável**? Ele é **facilmente entendível** por outras pessoas?

HOOKS NO REACT

São funções que permitem manipular recursos de forma simplificada.

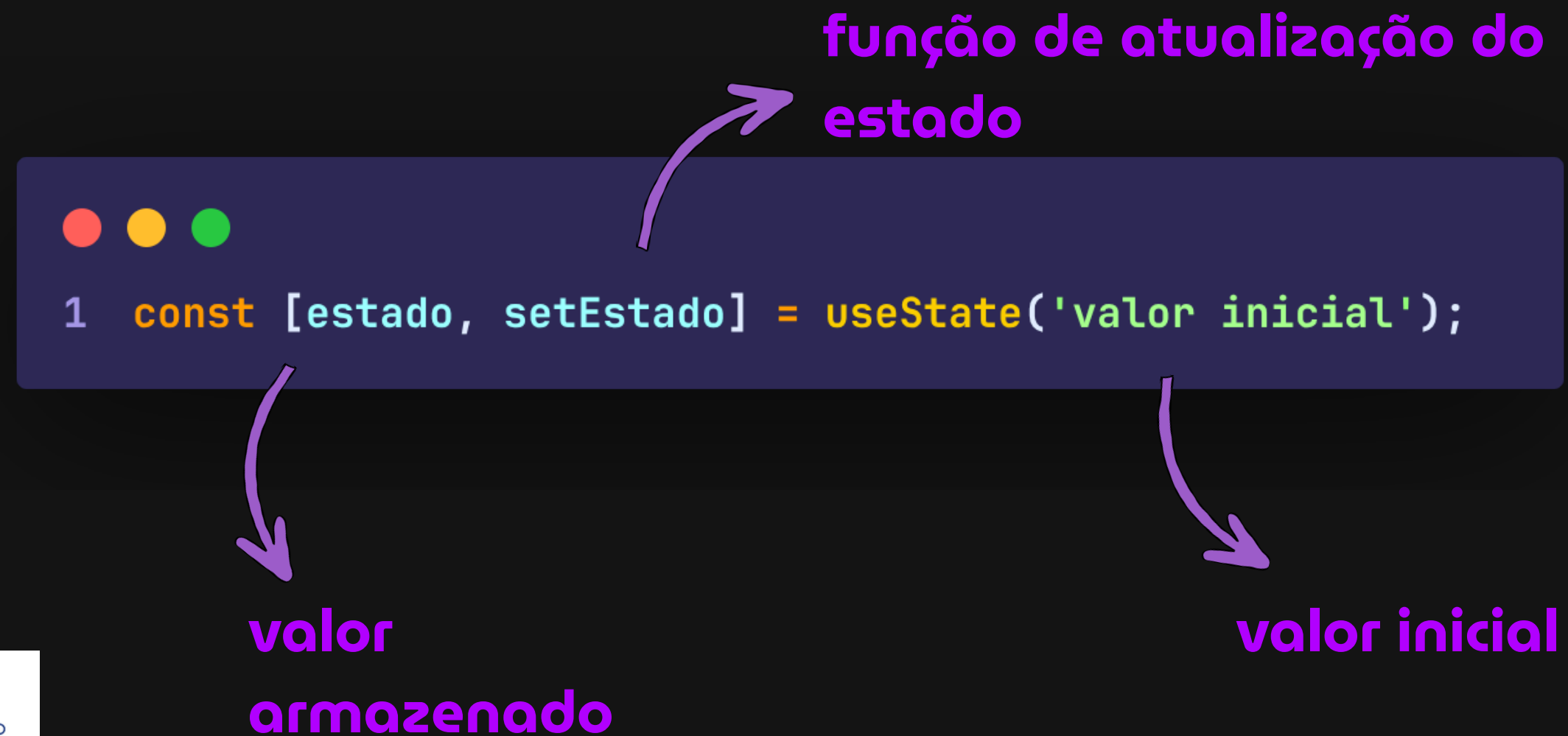
Na prática, os **hooks** servem para controlar a lógica dos componentes.

Se pensarmos em uma aplicação com a arquitetura MVC, os hooks podem ser interpretado como **Controllers**.

HOOKS NO REACT

useState

Serve para o componente armazenar algum valor, como uma variável.

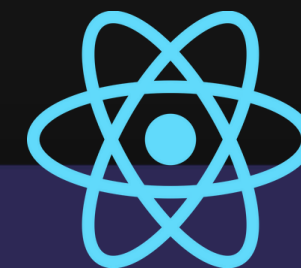


HOOKS NO REACT

useState



```
1 public class Medicamento {
2     private String nome;
3     private int dose;
4     private int intervalo;
5
6     public String getNome() {
7         return nome;
8     }
9
10    public void setNome(String nome) {
11        this.nome = nome;
12    }
13
14    public int getDose() {
15        return dose;
16    }
17
18    public void setDose(int dose) {
19        this.dose = dose;
20    }
21
22    public int getIntervalo() {
23        return intervalo;
24    }
25
26    public void setIntervalo(int intervalo) {
27        this.intervalo = intervalo;
28    }
29 }
```



TS

```
1 import { useState } from "react"
2
3 export const Medicamento = () => {
4     const [nome, setNome] = useState<string>();
5     const [dose, setDose] = useState<number>();
6     const [intervalo, setIntervalo] = useState<number>();
7
8
9     // renderização do componente
10 }
```

HOOKS NO REACT

useState

Se é como se fosse uma variável, por que não usamos variáveis nativas do JavaScript? (**let** ou **var**)

Essa atualização não dispara uma **renderização**. De fato o valor da variável é atualizado, mas o HTML não.

```
1 export const BotaoContador = ({ className }: BotaoContador) => {
2   let cliques = 0;
3
4   function atualizarCliques() {
5     cliques = cliques + 1;
6   }
7
8   const texto = `${cliques} clique${cliques !== 1 ? 's' : ''}`;
9
10  return (
11    <button onClick={atualizarCliques} className={className}>
12      {texto}
13    </button>
14  );
15 }
```

HOOKS NO REACT

useEffect

É utilizado para executar código que interage com o **"mundo exterior"**, como **buscar dados de uma API**, manipular o DOM diretamente ou configurar **timers**.

Ele roda após a renderização do componente.

HOOKS NO REACT

useEffect

```
1 import { useEffect, useState } from "react";
2
3 interface Character {
4   id: number;
5   name: string;
6   status: string;
7   species: string;
8   image: string;
9 }
```

```
1 export const CharacterData = () => {
2   const [character, setCharacter] = useState<Character | null>(null);
3
4   useEffect(() => {
5     // Exemplo de consumo da API do Rick and Morty
6     fetch('https://rickandmortyapi.com/api/character/1')
7       .then(res => res.json())
8       .then(data => setCharacter(data));
9   }, []);
10
11   if (!character) return <p>Carregando... </p>;
12
13   return (
14     <div>
15       <h2>Personagem: {character.name}</h2>
16       <img src={character.image} alt={character.name} width={150} />
17       <p>Status: {character.status}</p>
18       <p>Espécie: {character.species}</p>
19     </div>
20   );
21 }
```

HOOKS NO REACT

useEffect

Seu comportamento é controlado pelo array de dependências **(o segundo argumento)**.

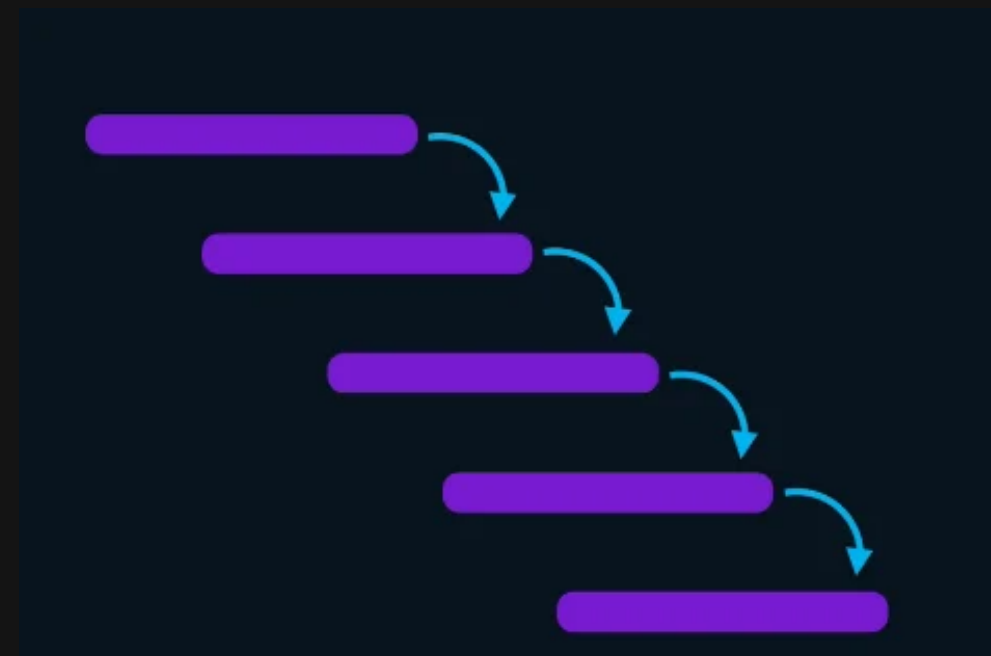
- **[] (array vazio)**: O efeito roda apenas uma vez, após a primeira renderização do componente. Ideal para buscar dados iniciais.
- **[valor1, valor2] (com valores)**: O efeito roda na primeira renderização e sempre que qualquer um dos valores dentro do array mudar, uma espécie de “observador”.
- **Sem array**: O efeito roda após toda e qualquer renderização. **(Use com cuidado!)**

HOOKS NO REACT

useContext

Serve para compartilhar estados e informações **globalmente** para todos os filhos.

Evita o **prop drilling**, passagem de propriedades ao longo da árvore de componentes.



HOOKS NO REACT

useContext

```
1 import { useState } from "react";
2 import { CartaoTema } from "../components/CartaoTema";
3
4 export default function PaginaTema() {
5   const [tema, setTema] = useState<"claro" | "escuro">("escuro");
6
7   const corFundo = tema === "claro" ? "#FFFFFF" : "#882cdd";
8
9   const trocarTema = () => {
10     if (tema === "claro") setTema("escuro");
11     else setTema("claro");
12   };
13
14   return (
15     <div
16       style={{
17         width: "100%",
18         height: "100dvh",
19         backgroundColor: corFundo,
20       }}
21     >
22       <button onClick={trocarTema}>Trocar tema</button>
23       <CartaoTema titulo="Tema Dinâmico" tema={tema} />
24     </div>
25   );
26 }
27
```

```
1 import { Informacoes } from "../Informacoes";
2
3 interface CartaoTemaProps {
4   titulo: string;
5   tema: "claro" | "escuro";
6 }
7
8 export const CartaoTema = ({ titulo, tema }: CartaoTemaProps) => {
9   return (
10     <
11       <h2>{titulo}</h2>
12       <Informacoes tema={tema} />
13     </>
14   );
15 }
16
```

```
1 interface InformacoesProps {
2   tema: "claro" | "escuro";
3 }
4
5 export const Informacoes = ({ tema }: InformacoesProps) => {
6   const dataHoje = Intl.DateTimeFormat("pt-BR", { dateStyle: "medium" }).format(
7     new Date(),
8   );
9   const corFundo = tema === "claro" ? "#FFFFFF" : "#882cdd";
10   const corData = tema === "claro" ? "#b100ff" : "#FFFFFF";
11
12   return (
13     <div style={{ padding: 10, backgroundColor: corFundo }}>
14       <p>Este é um componente de informações adicionais.</p>
15       <p style={{ color: corData }}>Hoje é dia {dataHoje}</p>
16     </div>
17   );
18 };
```

HOOKS NO REACT

useContext



```
1 // definindo o contexto
2 import { createContext } from "react";
3
4 type Tema = "claro" | "escuro";
5
6 export const TemaContext = createContext<Tema>("claro");
```

HOOKS NO REACT

useContext

```
1 // instanciando o contexto por meio do Provider
2 import { TemaContext } from "@app/context/TemaContext";
3
4 const MeuComponentePai = () => {
5   return (
6     <TemaContext.Provider value="escuro">
7       <MeuComponenteFilho />
8     </TemaContext.Provider>
9   );
10 };
```


HOOKS NO REACT

useContext



```
1  // usando o contexto no componente filho
2  import { useContext } from "react";
3
4  const temaAtual = useContext(TemaContext);
5
```


HOOKS NO REACT

useContext

não precisa mais
receber o tema como
prop!

```
1 import { useState } from "react";
2 import { CartaoTema } from "../components/CartaoTema";
3 import { TemaContext } from "../context/TemaContext";
4
5 export default function PaginaTema() {
6   const [tema, setTema] = useState<"claro" | "escuro">("escuro");
7
8   const corFundo = tema === "claro" ? "#FFFFFF" : "#882cdd";
9
10  const trocarTema = () => {
11    if (tema === "claro") setTema("escuro");
12    else setTema("claro");
13  };
14
15  return (
16    <TemaContext.Provider value={tema}>
17      <div
18        style={{
19          width: "100%",
20          height: "100dvh",
21          backgroundColor: corFundo,
22        }}
23      >
24        <button onClick={trocarTema}>Trocar tema</button>
25        <CartaoTema titulo="Tema Dinâmico" />
26      </div>
27    </TemaContext.Provider>
28  );
29 }
```

```
1 import { Informacoes } from "../Informacoes";
2
3 interface CartaoTemaProps {
4   titulo: string;
5 }
6
7 export const CartaoTema = ({ titulo }: CartaoTemaProps) => {
8   return (
9     <
10       <h2>{titulo}</h2>
11       <Informacoes />
12     </>
13   );
14 };
15
```

```
1 import { useContext } from "react";
2 import { TemaContext } from "../context/TemaContext";
3
4 export const Informacoes = () => {
5   const tema = useContext(TemaContext);
6
7   const dataHoje = Intl.DateTimeFormat("pt-BR", { dateStyle: "medium" }).format(
8     new Date(),
9   );
10   const corFundo = tema === "claro" ? "#FFFFFF" : "#882cdd";
11   const corData = tema === "claro" ? "#b100ff" : "#FFFFFF";
12
13   return (
14     <div style={{ padding: 10, backgroundColor: corFundo }}>
15       <p>Este é um componente de informações adicionais.</p>
16       <p style={{ color: corData }}>Hoje é dia {dataHoje}</p>
17     </div>
18   );
19 };
```

HOOKS NO REACT

useContext

Alguns exemplos de **context** bem comuns:

- TemaContext - diferentes temas;
- AuthContext - proteção de rotas baseada no perfil, verificação de usuário logado;
- PreferenciasUsuarioContext - personalização de experiência;
- NotificacaoContext - feedback global, mensagens de erro.

VAMOS PRATICAR!

Vamos adicionar hooks aos nossos componentes.

HOOKS NO REACT

custom

Podemos combinar os hooks que já conhecemos para criar hooks personalizados de acordo com a aplicação a ser desenvolvida.

Por convenção, todos os hooks são nomeados com o prefixo **use**.

HOOKS NO REACT

custom

Exemplo de hook para lista de itens editáveis:

```
1 import { useState } from "react";
2
3 export const useListaEditavel = <T>(listaInicial: T[]) => {
4   const [items, setItems] = useState<T[]>(listaInicial);
5
6   const adicionarItem = (item: T) => {
7     setItems((prev) => [...prev, item]);
8   };
9
10  const removerItem = (index: number) => {
11    setItems((prev) => prev.filter((_, i) => i !== index));
12  };
13
14  return {
15    items,
16    adicionarItem,
17    removerItem,
18  };
19 };
20
21 export type UseListaEditavelReturn<T> = ReturnType<typeof useListaEditavel<T>>;
```


HOOKS NO REACT

custom

```
1 import styles from './page.module.css';
2 import { useListaEditavel } from '@app/hooks/useListaEditavel';
3
4 export default function PaginaEquipamentos() {
5   const itensIniciais = ["Equipamento 1", "Equipamento 2", "Equipamento 3"];
6   const { items, removerItem, adicionarItem } =
7     useListaEditavel<string>(itensIniciais);
8
9   return (
10     <div>
11       <div>
12         <h1>Lista de equipamentos</h1>
13         <ul>
14           {items.map((item, index) => (
15             <li key={index}>
16               <span>{item}</span>
17               <button onClick={() => removerItem(index)}>Remover</button>
18             </li>
19           ))}
20         </ul>
21         <button
22           onClick={() =>
23             adicionarItem(
24               `Equipamentos ${parseInt(items[items.length - 1].split(" ")[1]) + 1}`,
25             )
26         >
27           + Adicionar equipamento
28         </button>
29       </div>
30     </div>
31   );
32 }
```

HOOKS NO REACT

custom

```
1 import styles from './page.module.css';
2 import { useListaEditavel } from '@app/hooks/useListaEditavel';
3
4 interface Medicamento {
5   nome: string;
6   dosagem: string;
7   intervalo: string;
8 }
9
10 const gerarNumeroAleatorio = (min: number, max: number) => {
11   return Math.floor(Math.random() * (max - min + 1)) + min;
12 };
13
14 export default function PaginaMedicamentos() {
15   const itensIniciais: Medicamento[] = [
16     { nome: "Medicamento 1", dosagem: "500mg", intervalo: "8h" },
17     { nome: "Medicamento 2", dosagem: "875mg", intervalo: "12h" },
18   ];
19
20   const { items, removerItem, adicionarItem } =
21     useListaEditavel<Medicamento>(itensIniciais);
22
23   const handleAdicionar = () => {
24     const nome = `Medicamento ${gerarNumeroAleatorio(1, 100)}`;
25     const dosagem = `${gerarNumeroAleatorio(100, 1000)}mg`;
26     const intervalo = `${gerarNumeroAleatorio(4, 24)}h`;
27     adicionarItem({ nome, dosagem, intervalo });
28   };
29
30   // ...
```

HOOKS NO REACT

custom

```
1  { /* ... */ }
2
3      <button onClick={handleAdicionar}>+ Adicionar Medicamento</button>
4
5      <ul>
6          {items.map((item, index) => (
7              <li key={index}>
8                  <div>
9                      <span>{item.nome}</span>
10                     <div>
11                         <span>{item.dosagem}</span>
12                         <span>A cada {item.intervalo}</span>
13                     </div>
14                 </div>
15                 <button onClick={() => removerItem(index)}>x</button>
16             </li>
17         ) ) }
18     </ul>
```

VAMOS PRATICAR!

No seu projeto, você consegue criar um hook
reutilizável e **coesivo**?

TAREFINHA DE CASA

- Experimentar **frameworks CSS** (Bootstrap, Tailwind CSS);
- Revisar **programação funcional** no React;
- Pesquisar outros **hooks populares** do React (useRef, useReducer, useMemo, useCallback);

O QUE VEM A SEQUIR?

- Manipulação de formulários;
- Consumo de APIs;
- Visualização de dados com gráficos;
- Mini POC (Prática final).

OBRIQADO!

